Artificial Intelligence (CS-203)

Objectives:

- **Knowledge as Production Rules**
- **Difference between Conventional and Rule based Programming**
- **Production rules in OPS-5 syntax**
- **Examples in OPS-5**

Knowledge as Production Rules

Operational knowledge can be better described in the form of production rules, which has been used for developing rule-based expert systems. Programming with the help of production rules can be termed as rule based programming.

This section introduces the underlying technology of developing rule-based system, which is a non-conventional software, although initiated in 1970s in healthcare domain but nowadays frequently available in healthcare domain. Systems developed using rule-based programming comprises of three basic components namely ***knowledge base*** (a collection of 'rules'), a ***working memory*** and ***inference engine***. Inference engines are those specified applications which have the capabilities to read a set of rules from the knowledge base, on input conditions identify the appropriate rule and execute the conforming actions. Rules are made up of if-then structures having diverse properties with respect to representation and computation. In rule-based programming, the term 'rule' (also known as 'production rule') refers to 'if-then' structure whereas 'if' represents condition and 'then' is followed by the action part.

IF <*condition*> THEN <*action*>

Conceptually, 'production rule' is the same as 'if' statement in conventional programming i.e. having condition portion and an action part. Action is performed when condition is true. However a major difference between a 'rule' and 'if-statement' is that a 'rule' is part of data, while 'if' statement is part of code. Changing a portion of code requires programming skill, recompilation of code, which leads to extremely high maintenance costs.

Difference between Conventional and Rule based Programming

In conventional programming, checks are encoded in the form of compiled codes written in some programming language like Java, C++, C#, etc. With new innovations in healthcare new checks and conditions are requisite to implement on the monthly or quarterly basis, which in turn may induce new bugs in the software; and excessive time is wasted to eradicate those bugs. In

conventional software 'if' statements are used (instead of production rules) as part of code and only persons with programming skills can change the code. Therefore, in case of conventional programming based software, professionals of healthcare domain are not able to incorporate updated knowledge in software by themselves. Some aspects of rule-based programming and conventional programming have been summarized in the Table 1 given below;

Table 1: Comparison of Rule-Based programming and conventional programming

Characteristics	**Rule Based Programming**	**Conventional Programming**
Type of Programming	Declarative	Procedural
Development Tools	Special Artificial intelligence languages (LISP, PROLOG) or in specific tools like OPS-5, KEE, SQL, Clips, Jess, Drools, ILOG Rules, G2. etc.	Compiled languages like Java, C++, C#, VB, etc. are used to develop conventional software.
Implementation of Conditions and Actions	Production rules	If-else statements, nested if-else statements, switch statement
Part of	Data	Code
System Specification	Flexible and dynamic	Complex, static
Processing zone	Centralized on database. Checks are executed on data stored in the database which updates the database directly.	Decentralized (mostly client-server model). Processing can be done on both client and server side.
Efficiency	More efficient because of direct implementation at the database.	Less efficient due to excessive network traffic
Database Server	No special requirements.	Server task is only to fetch and update data, so it depends upon size of data in the server.
User Friendliness	Easy to use and understand	Difficult to use and understand as software development skill and access code are necessary to update software

Production Rules in OPS-5 Syntax:

OPS-5 (Official Production System) was developed in late 1970's. It was written in LISP which uses "Forward Chaining Inference Engine".

In OPS-5 production rule look like this

(P production-name

LHS

➔

RHS

)

LHS contains a pattern describing working memory element

RHS is the action part

Example1: Area of Circle (Sequential Structure)

Write a program in OPS-5 which inputs radius of a circle from user then calculates and prints its area. Also trace working memory for this program.

Rule based Programming (OPS-5)

```
(P  inputrule
        (start)
        →
        (write "Enter Radius:")
        (read  <x>)
        (insert  'radius  <x>)
        (remove  1)
)
(P  calculateRule
        (radius  <r>)
        →
        (insert  'area  3.14* <r> * <r> )
        (remove 1)
)
(P  outputRule
        (area  <a>)
        →
        (write "Area of Circle=")
        (write <a>)
        (remove  1)
)
```

Conventional Programming (C++)

```
#include <iostream>
using namespace std;
int  input()
{       cout<<"Enter Radius:";
        cin>> x;
        return x;
}
float  calculate( )
{       int r;
        float a;
        r = input();
        a = 3.14 * r * r;
        return a;
}
void area( )
{       float a;
        a = calculate( );
        cout<< " Area of Circle ="<<a
                <<endl;
}
void main ( )
{       area( );
        system("pause");}
```

Command prompt (output window)

```
(insert  'start)
(run)

Enter Radius: 2
Area of Circle = 12.56
```

Working Memory

```
(start)
(radius 2)
(area 12.56)
```

Example2: (Conditional Structure)

Write a program in OPS-5 which inputs a number from user if the number is negative then make it positive and if the number is positive then make it double and in any case print the result. Also trace contents of working memory for this program.

```
(P   inputRule
        (begin)
        →
        (write "Enter a Number:")
        (read  <x>)
        (insert  'num    <x>)
        (remove 1)
)
(P   negativeRule
        (num   <x>  <  0)
        →
        (insert   'result   <x>*-1)
        (remove 1)
)
(P   positiveRule
        (num   <x>  >= 0)
        →
        (insert 'result <x>*2 )
        (remove 1)
)
(P   printRule
        (result   <r>)
        →
        (write "Resultant number =")
        (write <r>)
        (remove 1)
)
```

Command prompt (output window)

(insert 'begin)
(run)
Enter a number : 5
Resultant number = 10

Working Memory

(begin)
(num 5)
(result 10)

Command prompt (output window)

(insert 'begin)
(run)
Enter a number : -6
Resultant number = 6

Working Memory

(begin)
(num --6)
(result 6)

Example3: Printing Table (Repetition structure)

Write collection of production rules to input a number from user and display its multiplication table (first ten rows). Also show the status of working memory by tracing the rules if user enters 5 as input.

```
(P   inputRule
        (start)
        →
        (write "Enter a Number:")
        (read  <x>)
        (insert 'num   <x>   1)
        (remove 1)
)
(P   printRow
        (num   <x>   <i> <= 10)
        →
        (write  <x>   " * "  <i>  " = "   <x> * <i> )
        (insert 'num   <x>   <i>+1)
        (remove 1)
)
(P   endRule
        (num   <x>   <i> > 10)
        →
        (write  "END" )          //optional
        (remove 1)
)
```

Command Prompt (Output window)

(insert start)
(RUN)
Enter a Number: 5
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
END

Working Memory

(start)
(num 5 1)
(num 5 2)
(num 5 3)
(num 5 4)
(num 5 5)
(num 5 6)
(num 5 7)
(num 5 8)
(num 5 9)
(num 5 10)
(num 5 11)

Example 4: Calculating Factorial

Write collection of production rules to input a number from user and display its factorial. Also show the status of working memory by tracing the rules if user enters 4 as input.

```
(P  inputRule
        (start)
        →
        (write "enter a number:")
        (read  <x>)
        (insert  'num   <x>)
        (remove 1)                          )
(P   fact1
        (num   <x> == 1)
        →
        (insert  'fact  <x> )
        (insert  'factorial   1   1)
        (remove 1)       )
(P   fact2
        (num   <x> != 1)
        →
        (insert  'num  (<x>  - 1) )
        (insert  'fact  <x> )
        (remove 1)       )
(P   fact3
        (fact   <x> )
        (factorial  <x><y>)
        →
        (insert  'factorial  <x> + 1      <x> * <y> )
        (remove 1 ) (remove 2)  )
(P   result
        (factorial   <x><y>)
        - (fact  <x>)
        →
        (write  <x> - 1 " ! = " <y> )
        (remove 1))
```

Command prompt (output window)

```
(insert  'start)
(run)

enter number: 4
4 != 24
```

Working Memory

```
(start)
(num 4)
(num 3)
(fact      4)
(num 2)
(fact 3)
(num 1)
(fact 2)
(fact 1)
(factorial 1 1)
(factorial 2          1)
(factorial 3          2)
(factorial 4          6)
(factorial 5          24)
```

Following OPS code calculates factorial with less number of rules and more easy logic. Trace its execution yourself by showing working memory contents.

```
(P  inputRule
        (start)
        →
        (write "Enter a number:")
        (read  <x>)
        (insert  'num   <x> 1  1)
        (remove 1)                          )
(P   fact1
        (num   <x> <f> <i>  <= <x>)
        →
        (insert  'num  <x>   <x>*<i>     <i>+1 )
        (remove 1)                          )
(P   fact2
        (num   <x> <f> <i>  > <x>)
        →
        (write  <x>  " ! = " <f> )
        (remove 1)                           )
```